Indian Institute of Information Technology Dharwad

SELF-X: A Three-Timescale, Self-Improving Multi-Agent LLM Framework for Autonomous SDN Flow Management

Course Project — Problem Statement 1 (Version 3.0, revised to match the delivered implementation)

Field	Value	Field	Value
Course	EC466 — Software Defined Networks	Session	Aug – Nov 2026
Credits	3-1-0-0-4	Team ID	SDN_07
Instructor	Asst. Prof. Jagadeesha R Bhat (jagadeesha@iiitdwd.ac.in)	Office	F323, E-Block, 2nd Floor
Assessment	Assignments 30% • Project 50% • Viva 20%	Revision date	2026-11-09

Team and contribution

#	Name	Registration	Primary contribution
1	Ganesh Reddy J	23BDS024	Agent architecture, LoRA policy head, weight adaptation
2	M Jagadeswar	23BDS033	Evaluation harness, result persistence, data integrity
3	Harsh Raj	23BDS076	Dashboard, figure pipeline, manuscript
4	C Sai Aravind	23BDS075	Drift-plus-penalty derivation and backlog bound
5	Koushik C	23BCS073	Provider client, record-and-replay, reproducibility tooling
6	Dhruv R Katti	23BEC015	Traffic generator, datapath model, conservation checks

Why this revision exists

Version 1.0 of this document was written before any code existed, and its objectives were therefore aspirational. In November a line-by-line audit compared every claim against the repository and found that several did not hold: results reported in the draft paper matched no run that had been executed, the staged ablation was implemented as hard-coded behavioural stubs rather than the components it named, and a physical testbed was described that was never used. This version states the objectives the project actually pursued and marks each deliverable **Delivered**, **Partial** or **Dropped**. The audit trail is in **docs/REVISION_REPORT.md**.

1. Problem statement

A centralised SDN controller must make flow-management decisions that satisfy conflicting objectives — low delay, high goodput, bounded loss, low operating cost — while reacting to traffic that changes far faster than any human can retune it. Hand-crafted policies are brittle; value-based learners need thousands of interactions before they are useful and explore unsafely while they learn; and a large language model, which can reason over heterogeneous telemetry without a reward engineer, is far too slow to touch a congested datapath.

The quantitative form of that last obstacle is what this project is built around. A 10 Mbps path with a 2.0 Mbit buffer saturates in roughly 200 ms. One healthy free-tier language-model request, measured in this work, costs 3,191 ms. The model is therefore not merely slow, it is slower than the failure it is meant to prevent.

Research question. Not *can* a language model control a network, but **at which timescale does each mechanism earn its place**, and what does each one actually contribute once the others are present?

2. Objectives and delivery status

#	Objective	What was built	Status
O1	Decompose control into loops whose periods are set by the measured cost of the mechanism running in each, rather than by convention.	Three loops at 100 ms / 5 s / 30 s	Delivered
O2	Give the fast loop a provable backlog bound instead of an empirical one.	Lyapunov drift-plus-penalty, Theorem 1, bound $(B+VC)/\varepsilon$	Delivered
O3	Run a strictly incremental ablation: static, static LLM, +self-critique, +LoRA fine-tuning, +multi-agent governance, +fast loop.	Six stages, 5 paired seeds, identical offered demand	Delivered
O4	Drive every language-model decision from a real model, and record it so the study replays without a credential.	250 decisions, 100% served, 0 fallbacks	Delivered
O5	Keep every installed rule behind an independent deterministic safety arbiter that no learned component can reach.	selfx/safety.py; 20% drop ceiling, 512 kbps floor; vetoes counted	Delivered
O6	Internalise reflection experience into policy weights by LoRA fine-tuning, with a non-regression promotion gate.	$r=8$, $\alpha=16$ on q_proj/v_proj of a pinned SmolLM2-135M; disjoint train/gate seeds	Delivered
O7	Transfer the frozen policy zero-shot to an unseen workload, retaining ≥ 90% of its gain.	73% retained — below the 90% target; reported, not rounded up	Partial
O8	Evaluate on a live Mininet / Ryu testbed across three topologies.	Replaced by a bit-conserving fluid datapath model. Stated in the paper abstract, methods and limitations rather than implied away.	Dropped

3. System model as implemented

Two parallel logical paths join the ingress and egress switch of a linear four-switch OpenFlow 1.3 topology. A steady mice aggregate must stay on the short primary path; only the bursty elephant aggregate may be moved or metered. Time is slotted at τ; per slot and per path the fluid recursion clips the queue at the buffer, counts tail-drop overflow separately from meter drops, and conserves bits exactly. That identity is asserted on every slot of every run — the largest residual across the whole study is 8.9e-16 Mbit, which is floating-point noise.

Parameter	Value	Parameter	Value
Path capacity C■, C■	10 Mbps each	Fast slot τ	100 ms
Per-path buffer	2.0 Mbit	Control window	5 s
Propagation D■, D■	2 ms, 12 ms	Governance epoch	30 s
Actions	no_op, reroute, rate_limit	Lyapunov V (initial)	8.0
Safety floor / ceiling	512 kbps meter, 20% drop	Capacity slack ε	0.6
Seeds	1900, 1901, 1902, 1903, 1904	Windows per run	60

Reward (fixed before any run). R = goodput/10 – delay/50 – 1.5·loss/100 – 0.05·installs. Loss carries the largest weight because it is the failure the operator cares about most; the install term charges the controller for rule churn.

4. Methodology

- **Fast loop (τ).** A drift-plus-penalty optimiser whose cost model is the datapath's own one-slot recursion, so the optimisation is exact for the horizon it considers and any gap to optimal comes from the horizon, not from model mismatch.
- **Mid loop (window).** A LoRA-adapted policy head over a frozen, pinned SmolLM2-135M-Instruct, decoding constrained to three action tokens. It chooses an action and never a number: rates, ports and switch identifiers are synthesised by local arithmetic.
- **Slow loop (epoch).** An orchestrator and five specialists reach the free-tier model in one panel request and return only Δπ = (V, w, action mask). Deltas are clipped, weights renormalised, and a mask leaving fewer than two actions is rejected.
- **Charging the control plane for its own latency.** A proposal taking L seconds only takes effect after ■L/τ■ slots of its window, so a slow controller is penalised in the datapath rather than only in a side table.

5. Measured outcome

Metric	S0 static	S5 full pipeline	Change
Mean delay	96.9 ms	53.9 ms	–44.4%
Packet loss	32.2%	20.6%	–36.0%
Mean backlog	0.815 Mbit	0.488 Mbit	–40.1%
Mean window reward	-1.607	-0.549	+1.058
Decision cost	—	33.1 µs (fast loop)	vs 3,191 ms remote

Paired on 300 matched (seed, window) pairs: t = 11.66, p = 3.8e-26, Cohen's d = 0.67, seed-level cluster-bootstrap 95% CI [0.872, 1.218]. Because windows inside a seed are serially dependent, the bootstrap interval is the one relied on.

Honest accounting. Of the 32.2% loss under static forwarding, 8.0 percentage points are irreducible: superposed bursts push demand above the 20 Mbps total capacity in a minority of slots, where no work-conserving controller can avoid a drop. Only the remainder is credited to the controller.

6. Evaluation protocol and reproducibility

- **Baselines.** Static OpenFlow forwarding, a capacity-aware threshold heuristic, a greedy least-loaded (ECMP-style) selector, and a tabular Q-learner trained online on the same reward and action set.
- **Protocol.** 5 seeds × 60 windows × 50 slots. Within a seed every configuration sees bit-identical offered demand, hashed into the result manifest, so comparisons are paired at the (seed, window) level.
- **Synthetic data.** All traffic is generated from a seed by **selfx/traffic_gen.py**; nothing is captured or proprietary. Three profiles: steady_bursty (main study), flash_crowd (transfer hold-out), volatile_dpp (V sweep).
- **One command. scripts/run_all.ps1** (or run_all.sh) regenerates every result, figure, LaTeX macro and slide. It defaults to replay mode, which needs no API key and is deterministic.
- **No typed numbers.** The paper reads every value from a generated macro file, and **scripts/check_paper_macros.py** fails the build if a result is hard-coded in the manuscript.

7. Known limitations

- The datapath is a discrete-time fluid-queue model of a two-path OpenFlow topology. Queue integrals and bit conservation are exact; TCP dynamics, per-packet scheduling and real flow-mod installation times are not modelled. Relative ordering of controllers is well supported; absolute millisecond values are model outputs.
- The analytic backlog bound is loose in this regime — the 2 Mbit physical buffer binds long before the ceiling does.
- Zero-shot transfer retains 73% of the gain, below the 90% target set beforehand.
- One topology family and one workload generator; the free tiers the slow loop depends on change availability without notice, which is why the recorded trace rather than a live run is the reproducibility artefact.

Generated from **selfx/output/pipeline_study/** on 2026-11-09. Study run 2026-11-09T10:23:01.363500+00:00 in **live** mode. This document supersedes versions 1.0 and 2.0, which are retained unmodified.